ADAS-Ready Roads: Technical Report

OpenCV AI Competition 2026, powered by AWS · @anna · Oct 4, 2026

Summary

On four Sydney suburbs it had never seen, the system confirmed visible lane lines at 42 of 48 decided locations (88%, 95% CI 75 to 94%) and sent 7 more to human review. ADAS-Ready Roads is an agent that screens streets for lane marking visibility from crowdsourced Mapillary photos, so councils can find where driver assist cameras may struggle without sending out survey vehicles.

OpenCV 5 checks every photo first: a quality gate, semantic segmentation and a lane detection network, all running through the OpenCV DNN module. The agent then gathers more photos when evidence is thin, checks OpenStreetMap, asks a vision language model on Amazon Bedrock for a second look at the best photos, and routes split or unclear cases to people. Everything runs on AWS in Sydney: a public API on Lambda (ARM64) and a public map dashboard on S3 and CloudFront.

The main lessons came from honest evaluation. Three versions that used OpenCV alone looked good on development data and fell to chance level on unseen suburbs. A labelling study showed that even a person cannot grade paint wear reliably from crowd photos (kappa 0.14), so the system answers the measurable question instead: are painted lane lines visible? Detecting roads where markings are missing is not yet demonstrated, because such roads were rare in the data (16 of 133 labelled locations).

- **Live dashboard:** d1dzzj0gvxxl4o.cloudfront.net
- **Live API:** Lambda Function URL (POST a Mapillary image id or a photo; a browser visit shows the OpenCV version, models and git commit running on AWS)

To see it quickly, start with the **Quick tour** box at the top of the dashboard: one location and its evidence, one agent decision step by step, and one live photo analysed on AWS.

Problem and users

Lane keeping and lane departure systems steer by reading painted lane lines through a camera, so faded, missing or hidden markings quietly switch them off. Road owners rarely know which of their streets those cameras can read. Survey vehicles with calibrated cameras answer that, but they are expensive and cover each street rarely.

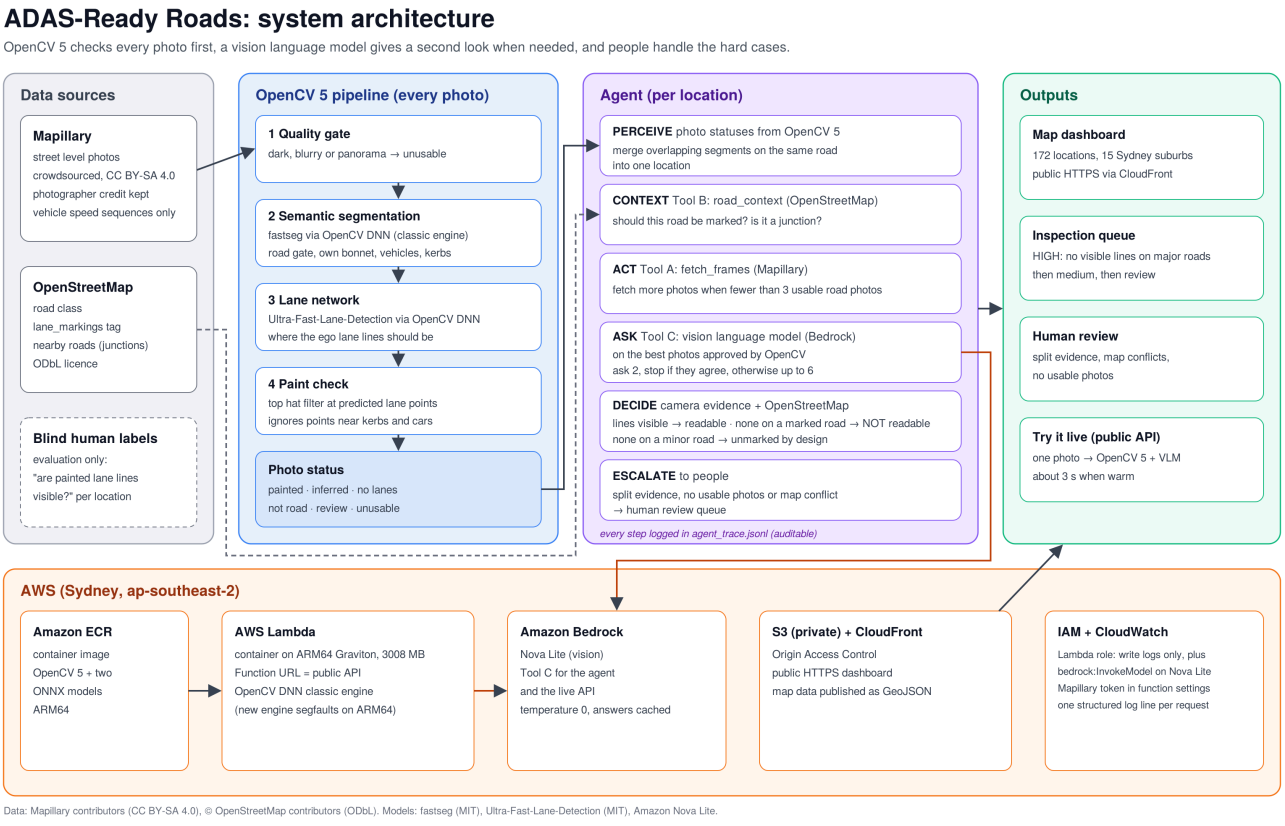
The intended users are local council road asset teams. They need a ranked list of places to inspect, not a certificate: the system screens, and people decide.

The question was reframed during the project. The first version tried to grade paint condition (ready, degraded, failing) from single photos. That grading overfitted the development photos and did not hold on new suburbs, and a second blind labelling pass showed that a person cannot grade wear consistently from crowd photos either. The final question is narrower and measurable: *are painted lane lines visible to a camera along this stretch of road?* Each answer is given per location, combining several photos, never from one photo alone.

Mapillary provides the photos: free, crowdsourced street level imagery under CC BY-SA 4.0. It is uneven in camera, time and coverage, which is exactly what a low cost screening tool has to cope with.

Architecture

The system has four stages: data sources, an OpenCV 5 pipeline that checks every photo, an agent that decides per location, and outputs for people, with all compute and hosting on AWS.



Mapillary photos enter the OpenCV 5 pipeline, which gives each photo a status. The agent reads those statuses, adds OpenStreetMap context, fetches more photos when needed, asks Amazon Bedrock about the best road photos, and either decides or escalates. Results feed a public map dashboard and an inspection queue, and a public API runs the same OpenCV 5 and Bedrock steps on any single photo.

OpenCV 5 implementation

Every photo passes through one OpenCV 5 pipeline (`adas_pipeline.py`, version v4) that runs in about 0.6 s on a laptop CPU and about 2 s on Lambda. Both neural networks run through the OpenCV DNN module from ONNX files, so the pipeline has no PyTorch dependency at run time.

1. **Quality gate.** Photos that are panoramas, too dark (mean grey below 50) or too blurry (variance of the Laplacian below 50) are marked unusable.
2. **Semantic segmentation.** fastseg MobileV3Large, trained on Cityscapes and exported to ONNX at 512 × 1024, labels every pixel. It gives the road gate (a photo with too little road or footpath in the lower view is marked not road), the vehicles, and the kerbs.
3. **Own bonnet removal.** Scanning up from the bottom of the image, rows that are mostly "vehicle" are the camera car's own bonnet or dashboard and are cut from the search area.
4. **Lane network.** Ultra-Fast-Lane-Detection (ResNet-18, CULane weights, ONNX at 288 × 800) predicts where the left and right ego lane lines should be, row by row.
5. **Paint check.** At each predicted lane point, a white top hat filter on the lightness channel tests whether paint is really there, ignoring points within 12 px of a kerb or vehicle. A lane counts as painted when at least 30% of its points carry paint, which suits dashed lines.

Each photo ends with one status: painted, inferred (lanes predicted but no paint), no lanes, not road, review or unusable. These statuses are what the agent perceives.

OpenCV 5 findings

- **The new DNN engine crashes on Linux ARM64.** With OpenCV 5.0.0, `net.forward()` on both ONNX models segfaulted every time inside the Lambda container, and occasionally on macOS. Single thread mode did not help. The classic engine (`cv.dnn.readNetFromONNX(path, engine=cv.dnn.ENGINE_CLASSIC)`) is stable at

about 250 to 310 ms per segmentation pass on ARM64, so the deployment uses it. On all 320 usable development photos, verdicts from the classic and new engines agreed on 318 (99.4%); the two differences sat on a decision threshold.

- **API change.** `HoughLinesP` returns a different array shape than in OpenCV 4, handled with `reshape(-1, 4)` in the earlier detector versions.
- **Safe model loading.** The lane network's PyTorch checkpoint was loaded with `weights_only=True` before export, so only tensors, never code, were read.

Agentic workflow

The agent (`agent.py`, v5) works per location, and what OpenCV sees decides what it does next: whether to fetch more photos, which photos to send to the vision language model, how many questions to ask, and whether to stop and hand over to a person.

Before the loop, it keeps only photos taken from a moving vehicle (sequences slower than 10 km/h are walking or handheld shots), chains each sequence into segments of up to 100 m, and merges overlapping segments on the same named road within 30 m into one location. On the final test suburbs this turned 107 segments into 48 locations, so one place is never counted several times.

Step	What happens	Tool
Perceive	Read the OpenCV 5 status of every photo at the location	pipeline results
Context	Road class, `lane_markings` tag and nearby road names (junction)	Tool B `road_context`, OpenStreetMap Overpass, cached, retried with back off
Act	Fewer than 3 usable road photos: fetch up to 8 more within about 45 m, then about 90 m, keeping only photos heading along the road	Tool A `fetch_frames`, Mapillary API, global budget of 60 calls
Ask	Send the best OpenCV approved road photos, one at a time: stop after 2 if they agree, otherwise ask up to 6	Tool C `ask_lines_visible`, Amazon Bedrock Nova Lite, cached

Step	What happens	Tool
Decide	Lines visible: readable. No lines on a road OSM expects to be marked: not readable (high priority on major roads). No lines on a minor road: unmarked by design	rules
Escalate	Split answers, no usable road photos, or lines seen where OSM says `lane_markings=no`: human review	review queue

The adaptive questioning averaged 2.3 to 2.5 questions per location, so the model sees few photos and only ones OpenCV has already accepted as road views. A single photo is never a verdict.

Observability. Every step of every location is written to `*_agent_trace.jsonl` (inputs, tool status, counts before and after, the decision and its reason), and every photo used is written to `*_agent_frames.csv` with its OpenCV status, the model's answer and the photographer's name. Contact sheets (`view_segments.py`) show every photo behind any location.

Example trace (clean test, location L0015, Waratah Lane). This trace from `final2_agent_trace.jsonl` shows OpenCV output changing what the agent did next:

1. **Perceive.** 4 photos, but OpenCV accepted only 2 as usable road views, and its paint check confirmed paint in none (painted rate 0.0).
2. **Context.** OpenStreetMap: a service lane, not expected to be marked, not a junction.
3. **Act.** With only 2 road photos (fewer than 3), the agent fetched 8 more photos within 44 m; all 8 passed the heading filter, giving 8 road photos.
4. **Ask.** It sent the best of the 8 to Nova Lite one at a time: 2 of 2 said lines visible, so it stopped after 2 questions instead of 6.
5. **Decide.** Readable: "painted lane lines visible (2 of 2 photos)", no priority.

Without the fetch, this location would have rested on 2 photos. Without the second look, OpenCV's 0.0 paint rate and OpenStreetMap's "unmarked" would both have suggested a road with no markings.

In the live API, a Kensington photo where the OpenCV lane network found no lanes was answered "lines visible" by the vision language model, agreeing with the earlier hand built detector, which had rated it ready. At Cottenham Avenue, in an earlier agent version,

OpenStreetMap tagged `lane_markings=no`, so the agent skipped fetching because more photos could not change the decision.

Human control. People see every flagged location with its reason and photo count ("no visible lane lines, 2 of 2 photos"), and decide. The dashboard labels all verdicts as screening hints.

Failure handling. Every tool failure has a defined outcome. OpenStreetMap queries retry up to 3 times with back off, failures are never cached, and a failed lookup leaves the road type unknown instead of guessing. If Bedrock refuses access, the agent stops cleanly and keeps every cached answer, so a rerun resumes where it stopped. In the live API, a model failure returns "not assessed" alongside the OpenCV result, never an error for the whole request. A Mapillary error or an empty fetch ends the fetch loop, and a budget of 60 fetch calls caps API use.

Task effort. On the 48 location final set the agent used 16 of its 60 fetch calls and asked 122 model questions (2.5 per location). On the 29 development locations it used 7 fetch calls and 68 questions.

Tested. Ten automated tests check the agent's decisions, with the model, Mapillary and image files replaced by test doubles: it stops after two matching answers, fetches only when road photos are too few, never sends photos OpenCV rejected to the model, routes split answers and map conflicts to a person, and marks missing lines on major roads as high priority.

AWS deployment

The whole system runs in the Sydney region (ap-southeast-2) on an AWS Free plan account, and the public endpoint answers a photo in about 3 s when warm.

Service	Role	Configuration
Amazon ECR	Stores the container image	OpenCV 5 headless, both ONNX models, ARM64
AWS Lambda	Single photo API (OpenCV 5 pipeline, then Bedrock)	Container image, ARM64 Graviton, 3008 MB, 60 s timeout
Lambda Function URL	Public HTTPS endpoint	Auth NONE, CORS for POST, `InvokeFunction` limited to calls via the URL

Service	Role	Configuration
Amazon Bedrock	Tool C, the second look	Nova Lite, temperature 0, max 5 tokens
Amazon S3	Dashboard files and map data	Private bucket
Amazon CloudFront	Public dashboard	Origin Access Control, HTTPS only, no cache headers on data
IAM	Least privilege	Lambda role may only write logs and call `bedrock:InvokeModel` on Nova Lite
CloudWatch Logs	Observability	One structured JSON line per request: status, model answer, source, latency

Measured latency. OpenCV took about 2.1 s and the model call about 0.9 s per photo when warm (3.0 s total). A cold start took about 16 s while the two models loaded. On the Mac the same OpenCV pipeline takes about 0.6 s.

Security. The Mapillary token lives in the function's settings, never in code or git. Error responses never reveal internal details. Uploads are capped at 6 MB, and Mapillary ids must be digits. The dashboard escapes every value before showing it.

Proof of what runs. Opening the API link in a browser returns a usage guide plus a runtime section read inside the running container: OpenCV 5.0.0, the classic DNN engine, both ONNX models with their size and SHA-256 fingerprint, the Bedrock model, the platform (Linux aarch64) and the git commit the image was built from.

Cost. As of 4 October 2026 the billing console showed $0.00 of credit used, with $120.00 remaining ($100 Free plan credit plus $20 earned for building a Lambda web app); AWS notes these estimates update about every 24 hours. Model calls were few (2.3 to 2.5 per location, cached), and Lambda, S3 and CloudFront usage stayed small.

Operations lessons. Lambda rejected Docker's default attestation manifests, so images are built with `--provenance=false --sbom=false`. New Function URLs also need `lambda:InvokeFunction` (restricted with `--invoked-via-function-url`), not only `InvokeFunctionUrl`. Returning a CORS header from the handler as well as the Function URL sent it twice, and browsers blocked the response. A second, accidental CloudFront distribution meant cache refreshes went to the wrong one; deployments are now checked by fetching what the live URL actually serves.

Evaluation

The final system confirms visible lane lines at about 86% of lined locations across three evaluations, while three earlier OpenCV only versions scored at chance on unseen suburbs. Every number below comes from blind human labels made on contact sheets of raw photos, with no verdicts or overlays shown.

Protocol

- Thresholds and rules were tuned on development data only; each test set was used once, and any reuse is disclosed.
- Fresh suburbs were downloaded before labelling, kept in separate folders and out of git.
- Contact sheets for labelling were generated from the exact run being evaluated, after an earlier run showed that location ids can shift between runs.
- Label reliability was measured with a second blind, shuffled labelling pass.
- Agreement is reported with 95% Wilson confidence intervals, Cohen's kappa, balanced accuracy, and two baselines: always the majority answer, and OpenStreetMap alone.

Label reliability

Two blind passes over the same 36 locations agreed on whether a road was marked 78% of the time (kappa 0.38), but on readable versus at risk only 62% (kappa 0.14, poor). Eight of the ten locations called at risk in the first pass were called readable in the second. Grading wear from crowd photos is therefore not a measurable target, and the final evaluation asks only whether painted lines are visible.

Version history

Version	Approach	Result
v0 to v3c	Hand built OpenCV paint detector (top hat, Hough lines, contrast threshold) with segmentation	Development photos (tuned on): 2 of 18 bad roads called ready. Locked photo test: 7 of 17. Fresh suburb location test: 42% agreement against a 50% baseline

Version	Approach	Result
v4	Ultra-Fast-Lane-Detection with paint verification	48 location final set: 44% agreement, kappa −0.01 (chance)
v5	OpenCV gated vision language model, adaptive evidence, OpenStreetMap	Same 48 locations: 78%, kappa 0.31. Clean test below

Photo review of v3c′s six high priority flags found that they were really two places, mostly junctions with roadworks, tram tracks, and walking photos of shop windows. That review produced the vehicle speed filter, location merging and junction handling. For v4, the painted lane rate overlapped between roads with and without visible lines (11 of 32 lined locations scored 0.00), so no threshold on it beat the majority baseline.

Final results (v5)

Evaluation	Locations	Lines visible, confirmed	No lines, found	Sent to review
Clean test: Penrith, Sutherland, Liverpool, Ryde (frozen v5)	56	42 of 48 decided, 88% (CI 75 to 94%)	0 of 1	7
Final set: Chatswood, Hurstville, Hornsby, Parramatta CBD, Bankstown (reused, disclosed)	48	31 of 38, 82%	4 of 7	3
Development: Burwood, Bondi Junction	29	17 of 19, 89%	5 of 8	2
Pooled	133	90 of 105, about 86%	9 of 16, about 56%	12

On the clean test, both labelling passes agreed on 54 of 56 locations, and on those 54 the result was unchanged (42 confirmed, 6 to review, 6 wrongly "no lines"). On the 48 location set, balanced accuracy was about 69% against 50% for always saying yes, and OpenStreetMap alone reached only 54% (kappa −0.05).

Failure cases

- **False "no lines" (6 of 54 on the clean test).** These send a well marked road to inspection: a false alarm, not a dangerous miss.
- **Missing markings.** Only 16 of 133 labelled locations had no visible lines, and the system found 9. That is too few to claim the capability.
- **Complex scenes** (junctions, roadworks, tram tracks, glare) defeated every OpenCV only version; v5 routes split evidence to people.
- **OpenStreetMap outages.** 7 of 56 lookups failed twice on the clean test. In v5 this affects priority and grouping, not the lines visible verdict.

What OpenCV contributes, measured

Two later experiments measured how much the OpenCV 5 stage adds on top of the vision language model; its effect on final accuracy turned out to be small. Both used the same photos, labels and decision rules as the evaluated system, on reused or development data, and changed nothing in it.

Experiment	Result
Gate ablation (`ablation_gate.py`): every photo goes to the model, with no OpenCV filtering or ranking	Clean test 41 of 50 (82%) against 42 of 49 (86%) gated; final set 33 of 44 (75%) against 35 of 45 (78%), balanced 72% against 69%. Without the gate, 5% and 12% of model calls went to photos OpenCV rejects
Road focus probe (`crop_probe.py`): the model sees only the road region found by segmentation, or the road with everything else grey	Balanced accuracy 77% (crop) and 74% (grey) against 78% for the full photo, on 72 development locations. A rule fixed in advance required +5 points, so neither was adopted

In v5, OpenCV earns its place by deciding when the agent must fetch more evidence, choosing which photos the model sees, and avoiding wasted calls, not by changing accuracy much. The vehicle speed filter already removes most unusable photos, and the model judges best with the whole scene, probably using kerbs, road width and traffic as context.

Limitations, responsible use and future work

The system is a screening aid that ranks places for human inspection; it must never be used as a safety assessment or as proof that a road suits driver assistance.

Limitations

- Detection of missing markings is not validated: too few unmarked roads appeared in the data.
- All labels come from one labeller, and paint wear cannot be judged reliably from crowd photos.
- Crowd imagery varies in date, camera, weather and coverage; a verdict reflects when its photos were taken, not today.
- The vision language model is a third party model; its answers are cached and run at temperature 0, but a model update could change them.
- The lane network was trained on CULane (right hand traffic, research licence weights); its own detections are logged but no longer decide verdicts.
- Several queue flags rest on only 1 or 2 photos. The reason text shows the count so reviewers can weigh it.

Responsible use

- The dashboard and the API label every verdict as a screening hint, and the dashboard shows a validated performance panel that states what has not been demonstrated.
- Uncertain cases go to people instead of being guessed.
- Photographer credit and licence travel with every photo; no Mapillary images are stored in the repository.
- Pedestrian and handheld sequences are filtered out, which also avoids analysing people's walking routes.
- Least privilege IAM, no secrets in code, and generic error messages limit the public endpoint's exposure.

Future work

- Collect a deliberate sample of unmarked and badly worn roads, labelled by two people, to test missing marking detection properly.
- Compare against a council's own survey records where available.
- Add time awareness: prefer recent photos and flag streets whose photos are old.

- Benchmark the Cloud Optimized OpenCV Library (COOL) on Graviton against the current OpenCV build.

Reproducibility, credits and licences

The repository README gives the full build, run and deploy steps; dependencies are pinned in `requirements.txt`, `requirements-export.txt` (model export only) and `requirements-lock.txt` (exact versions used).

1. Create a Python 3.12+ virtual environment and install `requirements.txt` (OpenCV 5.0.0, NumPy, Requests, boto3).
2. Export the two ONNX models once with `export_seg.py` and `export_ufld.py`.
3. Set `MAPILLARY_TOKEN` and sign in to AWS (`aws login --region ap-southeast-2`).
4. Run `fetch_mapillary.py`, `run_pipeline.py` and `agent.py`; outputs are the location CSV and GeoJSON, the review queue, the trace and the frame log.
5. Rerun any evaluation with `evaluate_binary.py <labels> <system csv>`; the labels and system outputs for every test are committed.
6. Deploy by building the ARM64 container, pushing it to ECR and updating the Lambda; the dashboard is a static folder synced to S3.

Automated tests. `make check` runs 33 tests offline in about one second: the OpenCV quality gate and bonnet detection, segment building and the walking filter, the agent's decision rules, the evaluation maths (it reproduces the reported 75 to 94% confidence interval), and the API handler, including that unusable photos never reach Bedrock. `make trace` prints the Waratah Lane agent trace.

Model answers are cached in `vlm_cache.json`, so reruns over the same photos give the same verdicts. The complete evaluation log, including every disclosed reuse and failed lookup, is in `notes/final_evaluation.md`.

Credits and licences

- Street imagery: Mapillary contributors, CC BY-SA 4.0; the creator is recorded for every photo.
- Road data and map tiles: © OpenStreetMap contributors, ODbL.
- fastseg MobileV3Large (MIT), Cityscapes trained weights.
- Ultra-Fast-Lane-Detection, Qin, Wang and Li, ECCV 2020 (MIT), CULane trained weights.

- Amazon Nova Lite through Amazon Bedrock.
- Built with OpenCV 5, AWS Lambda, ECR, S3, CloudFront, IAM, CloudWatch and Bedrock.